PERSONAL REFERENCE

Vim Notes

Organised notes from hands-on Vim practice: modes, moving around, deleting, copy and paste, search and replace, macros, and working with several files. Every command here was checked in Vim 9.1.

01 Modes
02 Save and quit
03 Entering Insert mode
04 Moving around
05 Delete (cut)
06 Copy (yank) and paste
07 Change, undo, redo
08 Visual mode
09 Search
10 Replace
11 Macros (recording)
12 Indenting
13 Files, buffers, tabs and splits
14 Settings and .vimrc
15 Linux commands from inside Vim
16 Corrections to my notes

01 Modes

MODE	HOW TO GET THERE	USED FOR
Normal	`Esc`	Moving, deleting, copying. Vim starts here
Insert	`i`, `a`, `o` and friends (section 3)	Typing text
Visual	`v` characters, `V` lines, `Ctrl+v` block	Selecting text
Command-line	`:`	Save, quit, replace, settings

> Lost or stuck? Press `Esc` (twice if needed) to get back to Normal mode.

02 Save and quit

COMMAND	WHAT IT DOES
`:w`	Save, stay in Vim
`:w filename`	Save under a new file name
`:wq`	Save and quit
`:q`	Quit (only works if there are no unsaved changes)
`:q!`	Force quit: throw away unsaved changes
`:wa`	Save all open files
`:wqa`	Save all open files and quit
`:xa`	Save the changed files and quit, without prompts

03 Entering Insert mode

KEY	START TYPING...
`i`	before the cursor
`I`	at the start of the line (first non-blank character)
`a`	after the cursor
`A`	at the end of the line
`o`	on a new line below
`O`	on a new line above

04 Moving around

KEY	MOVES TO
`h` `j` `k` `l`	left, down, up, right
`w`	start of the next word (right)
`b`	start of the previous word (left)
`e`	end of the word
`0`	first column of the line, leading spaces included
`^`	first non-blank character of the line
`$`	end of the line
`gg`	first line of the file
`G`	last line of the file
`5G`, `5gg` or `:5`	line 5
`%`	the matching bracket: `( )`, `[ ]`, `{ }`
`gd`	local declaration of the word under the cursor (inside the function)
`gD`	global declaration (searched from the top of the file)
`ma`	set mark a at the cursor
`'a`	back to the line of mark a
`` `a ``	back to the exact position of mark a

> `0` vs `^`: on the line `int count = 10;`, `0` lands on the first space and `^` lands on the `i`.

05 Delete (cut)

Deleting in Vim is cutting: the deleted text is kept, and `p` pastes it back.

COMMAND	DELETES
`x`	one character
`dd`	the current line
`5dd`	5 lines, starting at the cursor
`dw`	from the cursor to the start of the next word
`daw`	the whole word, wherever the cursor is inside it
`db`	back to the start of the word (the character under the cursor stays)
`D`	from the cursor to the end of the line
`dj` / `d2j`	the current line and 1 / 2 lines below
`dk` / `d2k`	the current line and 1 / 2 lines above
`d5gg`	from the current line up (or down) to line 5
`dgg` / `dG`	from the current line to the top / bottom of the file

By line number, from the `:` prompt:

COMMAND	DELETES
`:5d`	line 5
`:5,10d`	lines 5 to 10
`:1,3d`	the first 3 lines
`:$d`	the last line
`:%d`	every line
`:g/^$/d`	every blank line

> To delete lines 2 and 5, run `:5d` first, then `:2d`. If you delete line 2 first, the old line 5 moves up to line 4 and you delete the wrong line.

Handy: `ddp` swaps a line with the one below it; `xp` swaps two characters (`teh` becomes `the`).

06 Copy (yank) and paste

COMMAND	WHAT IT DOES
`yy`	copy the line
`yw`	copy from the cursor to the start of the next word
`y$`	copy from the cursor to the end of the line
`p`	paste after the cursor
`P`	paste before the cursor

Where the paste lands depends on what was copied:

COPIED WITH	`p` PUTS IT	`P` PUTS IT
Whole lines: `yy`, `dd`, `V`	on a new line below	on a new line above
Part of a line: `yw`, `y$`, `x`, `v`	right of the cursor	left of the cursor

Registers (named clipboards):

COMMAND	WHAT IT DOES
`"xyy`	copy the line into register x
`"xp`	paste from register x
`:reg`	show what is in every register
`:%y+`	copy the whole file to the system clipboard (needs a Vim built with clipboard support)

07 Change, undo, redo

COMMAND	WHAT IT DOES
`cw`	change from the cursor to the end of the word: type the new text, then `Esc`
`u`	undo
`Ctrl+r`	redo what was undone
`.`	repeat the last change

08 Visual mode

KEY	SELECTS
`v`	characters
`V`	whole lines
`Ctrl+v`	a rectangular block
`ggVG`	the whole file

With text selected: `y` copies, `d` deletes, `c` changes, `p` replaces the selection with what you copied.

> Doubt answered: paste works with Visual mode. Select with `v`, press `y`, move, press `p`. Mouse dragging only selects if mouse support is on (`:set mouse=a`); the keys are faster anyway.

Block editing with `Ctrl+v`:

KEYS	RESULT
`Ctrl+v` `jj` `I` `//` `Esc`	`//` added at the start of 3 lines (comment them out)
`Ctrl+v` `jj` `e` `c` new text `Esc`	the first word of 3 lines changed at once
`Ctrl+v` `jj` `$` `A` text `Esc`	text added at the end of 3 lines, even if they are different lengths

> As my notes say, `Ctrl+v` + `e` makes a ragged selection when the words have different lengths, so `c` only works cleanly when they are the same length.

09 Search

COMMAND	FINDS
`/word` then `Enter`	the next match, searching forward
`?word` then `Enter`	the previous match, searching backward
`n` / `N`	next / previous match
`/^Vim`	lines that start with Vim
`/\<Vim\>`	Vim as a whole word, not inside GVim
`/vim\c`	vim, ignoring case for this search only
`/Vim\C`	Vim, case-sensitive for this search only

Pattern pieces: `\<` start of a word, `\>` end of a word, `\c` ignore case, `\C` match case, `\&` both patterns must match.

SETTING	EFFECT
`:se hls` / `:se nohls`	highlight all matches on / off
`:se ic` / `:se noic`	ignore case on / off

10 Replace

COMMAND	REPLACES
`:%s/old/new/g`	every match on every line
`:5,10s/old/new/g`	only on lines 5 to 10
`:%s/old/new/gc`	every match, asking before each one
`:%s/\<old\>/new/g`	whole words only

- `%` means every line. (In Normal mode, `%` means something else: jump to the matching bracket.)
- `g` means every match on the line; without it, only the first match on each line changes.

11 Macros (recording)

1. `qa`: start recording into register a. The bottom of the screen shows `recording @a`.
2. Do the edit once.
3. `q`: stop recording.
4. `@a` plays it once, `3@a` plays it 3 times, `@@` plays the last macro again.

> Doubt answered: `3@b` replays exactly the keys you recorded. If the macro does not move to the next line, all 3 runs happen on the same line. Start the macro with `0` (start of line) and end it with `j` (down one line), so each run continues on the next line.

Example: turn a list of signal names into SystemVerilog declarations. On `clk`, type `qa` `0` `i` `logic` `Esc` `A` `;` `Esc` `j` `q`, then `3@a` for the other three lines:

```
clk             logic clk;
rst_n     -->   logic rst_n;
valid           logic valid;
ready           logic ready;
```

12 Indenting

COMMAND	WHAT IT DOES
`gg=G`	re-indent the whole file: `gg` go to the top, `=` re-indent, `G` down to the end

13 Files, buffers, tabs and splits

COMMAND	OPENS THE FILE	NEW TAB?	NEW WINDOW?
`:e notes.txt`	in the current window	No	No
`:tabnew notes.txt`	in a new tab	Yes	Yes, in that tab
`:new notes.txt`	in a horizontal split	No	Yes
`:sp notes.txt`	in a horizontal split	No	Yes
`:vsp notes.txt`	in a vertical split	No	Yes

Tabs:

COMMAND	WHAT IT DOES
`gt` or `:tabnext`	next tab
`gT` or `:tabprevious`	previous tab
`:tabclose`	close the current tab

Buffers (every file you have opened is a buffer, even when it is not on screen):

COMMAND	WHAT IT DOES
`:ls`	list all buffers
`:b 2`	switch to buffer 2
`:b notes.txt`	switch to notes.txt
`:bn` or `:bnext`	next buffer (not next tab)
`:bp` or `:bprev`	previous buffer
`:bd`	remove the current buffer from the list

Windows (splits):

KEYS	WHAT IT DOES
`Ctrl+w w`	move to the next window
`Ctrl+w c`	close the current window. If it was the only window in its tab, the tab closes too
`Ctrl+w o` or `:only`	close every other window

`:E` opens a file explorer for the current directory: move with the arrow keys, `Enter` opens a file.

14 Settings and .vimrc

COMMAND	WHAT IT DOES
`:se nu` / `:se nonu`	line numbers on / off
`:set relativenumber`	number each line by its distance from the cursor, handy for `5dd` or `d3j`
`:set norelativenumber`	back to normal line numbers

Settings typed with `:` are lost when Vim closes. To keep one, add it to `~/.vimrc`:

```
echo "set number" >> ~/.vimrc
```

> The `>` signs in my notes were the terminal's heredoc prompt, not part of the file. Use `>>` to append: `cat > ~/.vimrc` wipes out any settings already there.

15 Linux commands from inside Vim

COMMAND	WHAT IT DOES
`:!pwd`	run a shell command, then press Enter to return to Vim
`:!ls -l`	list files without leaving Vim

Linux bonus from the same session: delete every file in the current folder except lesson1.txt.

```
find . -maxdepth 1 -type f ! -name "lesson1.txt"          # check the list first
find . -maxdepth 1 -type f ! -name "lesson1.txt" -delete   # then delete (no undo)
```

16 Corrections to my notes

MY NOTES SAID	WHAT IS CORRECT
Paste in visual mode? (doubt)	Works: select with `v`, `y` to copy, then `p`
`3d` deletes 3 lines	`3d` waits for a motion and deletes nothing. Use `3dd`
`V` is block-wise visual	`V` is line-wise. Block-wise is `Ctrl+v`
`y` for paste	`y` copies (yank). `p` pastes
`p` pastes one line after the cursor	Only for whole lines. A copied word goes right of the cursor
`i` inserts at the cursor	Before the cursor. `a` is after
Macro: `qb`, edit, then `b` to stop	`qb` starts, `q` stops
`3@b` doubt	It repeats on the same line unless the macro ends with `j`
`GD` for global declaration	`gD` (small g). `GD` jumps to the last line and erases it
`Gg=G`	`gg=G`
"exponent" goes to the start of the line	That key is `^` (caret): first non-blank character
`d2gg` deletes 3 lines above	It deletes from the cursor up to line 2: `d{N}gg` deletes to line N
`Ctrl+w c` closes the tab	It closes the current window. The tab closes only if that was its last window
`:waq`	Not a Vim command. It is `:wqa`
`cat > ~/.vimrc`	Overwrites the file. Use `>>`